

LAB-9.3

Task Description#1 Basic Docstring Generation

- Write python function to return sum of even and odd numbers in the given list.
- Incorporate manual **docstring** in code with Google Style
- Use an AI-assisted tool (e.g., Copilot, Cursor AI) to generate a docstring describing the function.
- Compare the AI-generated docstring with your manually written one.

```
if __name__ == "__main__":  
    user_input = input("Enter a list of integers separated by spaces: ")  
    numbers = [int(x) for x in user_input.split()]  
    even_sum, odd_sum = sum_even_odd(numbers)  
    print(f"Sum of even numbers: {even_sum}")  
    print(f"Sum of odd numbers: {odd_sum}")
```

```

def sum_even_odd(numbers):

    even_sum = sum(n for n in numbers if n % 2 == 0)
    odd_sum = sum(n for n in numbers if n % 2 != 0)
    return even_sum, odd_sum


if __name__ == "__main__":
    user_input = input("Enter a list of integers separated by spaces: ")
    numbers = [int(x) for x in user_input.split()]
    even_sum, odd_sum = sum_even_odd(numbers)
    print(f"Sum of even numbers: {even_sum}")
    print(f"Sum of odd numbers: {odd_sum}")

    def sum_even_odd(numbers):
        """
        Returns the sum of even and odd numbers in the given list.

        Args:
            numbers (list of int): List of integers to process.

        Returns:
            tuple: A tuple containing two integers:
                - The sum of even numbers.
                - The sum of odd numbers.

        Example:
            >>> sum_even_odd([1, 2, 3, 4])
            (6, 4)
            """
        even_sum = sum(n for n in numbers if n % 2 == 0)
        odd_sum = sum(n for n in numbers if n % 2 != 0)
        return even_sum, odd_sum

```

OUTPUT:

```

.venv/Scripts/Activate.ps1
(.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8> & "C:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/lab-8/.venv/Scripts/python.exe" "c:/Users/Rishitha Reddy/OneDrive/Desktop/AIAC/lab-9/Task1.py"
Enter a list of integers separated by spaces: 2 43 4 5 6 7 7 8 8
Sum of even numbers: 28
Sum of odd numbers: 62
(.venv) PS C:\Users\Rishitha Reddy\OneDrive\Desktop\AIAC\lab-8>

```

Manual documentation:

- ☐ Define function purpose → find sum of even and odd numbers.
- ☐ Input → list of integers.
- ☐ Output → tuple (even_sum, odd_sum).
- ☐ Process → check each number with % 2 and add accordingly.
- ☐ Manual docstring (Google Style) → more detailed, with example.
- ☐ AI docstring → shorter, less structured.

Task Description#2 Automatic Inline Comments

- Write python program for **sru_student** class with attributes like name, roll no., hostel_status and **fee_update** method and **display_details** method.
- Write comments manually for each line/code block
- Ask an AI tool to add inline comments explaining each line/step.
- Compare the AI-generated comments with your manually written one.

```
class sru_student:
    def __init__(self, name, roll_no, hostel_status):
        self.name = name
        self.roll_no = roll_no
        self.hostel_status = hostel_status
        self.fee_paid = False

    def fee_update(self, status):
        self.fee_paid = status

    def display_details(self):
        print(f"Name: {self.name}")
        print(f"Roll No.: {self.roll_no}")
        print(f"Hostel Status: {self.hostel_status}")
        print(f"Fee Paid: {'Yes' if self.fee_paid else 'No'}")

if __name__ == "__main__":
    name = input("Enter student name: ")
    roll_no = input("Enter roll number: ")
    hostel_status = input("Enter hostel status (Hosteller/Day Scholar): ")
    student = sru_student(name, roll_no, hostel_status)
    fee_status = input("Has the fee been paid? (yes/no): ").strip().lower()
    student.fee_update(fee_status == "yes")
    student.display_details()
```

```

task2.py > sru_student > display_details
1  # Define a class named sru_student
2  class sru_student:
3      # Initialize the attributes: name, roll_no, hostel_status
4      def __init__(self, name, roll_no, hostel_status):
5          self.name = name # Store student's name
6          self.roll_no = roll_no # Store student's roll number
7          self.hostel_status = hostel_status # Store hostel status (True/False)
8          self.fee_paid = False # Initialize fee status as unpaid
9
10     # Method to update fee status
11     def fee_update(self, status):
12         self.fee_paid = status # Update fee status
13
14     # Method to display student details
15     def display_details(self):
16         print(f"Name: {self.name}") # Print student's name
17         print(f"Roll No: {self.roll_no}") # Print student's roll number
18         print(f"Hostel Status: {self.hostel_status}") # Print hostel status
19         print(f"Fee Paid: {self.fee_paid}") # Print fee status
20
21     # Create an instance of sru_student
22     student1 = sru_student("Alice", 101, True) # Create student with name, roll no, hostel status
23
24     # Update fee status for student1
25     student1.fee_update(True) # Mark fee as paid
26
27     # Display details of student1
28     student1.display_details() # Show all details of student1

```

OUTPUT:

```

Enter student name: ravi
Enter roll number: 13
Enter hostel status (Hosteller/Day Scholar): hosteller
Has the fee been paid? (yes/no): yes
Name: ravi
Roll No.: 13
Hostel Status: hosteller
Fee Paid: Yes

```

Manual documentation: **Create a class sru_student** with attributes name, roll_no, and hostel_status.

Initialize attributes inside the constructor (`__init__`) and set fee_paid default to False.

Define fee_update() method → updates fee status (True or False).

Define display_details() method → prints all student details.

Create object of sru_student class and call methods to update fee and display details.

Task Description#3

- Write a Python script with 3–4 functions (e.g., calculator: add, subtract, multiply, divide).
- Incorporate manual **docstring** in code with NumPy Style
- Use AI assistance to generate a module-level docstring + individual function docstrings.
- Compare the AI-generated docstring with your manually written one.

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    """
    Divides two numbers and returns the result.

    Args:
        a (float or int): The numerator.
        b (float or int): The denominator.

    Returns:
        float: The result of dividing a by b.

    Raises:
        ValueError: If b is zero.
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b

if __name__ == "__main__":
    x, y = 10, 5
    print("Add:", add(x, y))
    print("Subtract:", subtract(x, y))
    print("Multiply:", multiply(x, y))
    print("Divide:", divide(x, y))
```

```

task3.py > divide
1  def add(a, b):
2      return a + b
3
4  def subtract(a, b):
5      return a - b
6
7  def multiply(a, b):
8      return a * b
9
10 def divide(a, b):
11     """
12     Divides two numbers and returns the result.
13
14     Args:
15         a (float or int): The numerator.
16         b (float or int): The denominator.
17
18     Returns:
19         float: The result of dividing a by b.
20
21     Raises:
22         ValueError: If b is zero.
23     """
24     if b == 0:
25         raise ValueError("Cannot divide by zero.")
26     return a / b
27 if __name__ == "__main__":
28     x, y = 10, 5
29     print("Add:", add(x, y))
30     print("Subtract:", subtract(x, y))
31     print("Multiply:", multiply(x, y))
32     print("Divide:", divide(x, y))
33

```

OUTPUT:

```

C:/lab-9/task3.py
Add: 15
Subtract: 5
Multiply: 50
Divide: 2.0

```

Manual Documentation:

Define module docstring describing purpose of the calculator.

Create four functions → add, subtract, multiply, divide.

Write NumPy Style docstrings manually:

- Parameters (with types)
- Returns (with type)
- Raises (for divide only)
- Example usage

Use AI tool to generate docstrings → shorter and simpler.

Compare → Manual version is more detailed and standard; AI's version is concise but less formal.